

Last modified on

Sports Data Soccer API - Seasonal Stats Feed (TM4)

Get cumulative performance statistics for every player that has made an appearance in the specified tournament calendar.

Overview

The Soccer **Seasonal Stats feed** contains cumulative performance statistics for every player who has made an appearance in the specified tournament calendar. You can see a complete list of available **contestant statistics** and **player statistics** in this guide.

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY						
	TIER 15	TIER 12+14	TIER 4	TIER 11+13	TIER 8+9+10	TIER 6+7	TIER 5
Updated at Full Time in each game, with feeds delivered for the two teams involved in that match only - this is usually within 10 minutes of the game finishing	See Tier 13	Post-match	n/a	Once every day at the end of the given match day			n/a

Seasonal Stats (TM4) Guide

USAGE	FEED
GET seasonal statistics according to specified parameter values	/soccerdata/seasonstats/{outletAuthKey}?{queryParams}

If your request URL is as follows (does not contain either `tmc1` or `comp` and `ctst` parameters and UUIDs in the query criteria), the feed returns an error: `/soccerdata/seasonstats/{outletAuthKey}`

Query Parameters And Example Request Parameters

This feed also makes use of the [Global query parameters](#), and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/seasonstats` feed by a specific UUID of a tournament calendar, contestant or competition, and you can combine additional query parameters in a request and you can combine query parameters in your requests.

Legacy IDs: You can combine parameters for `comp` and `ctst` , or `tmc1` and `ctst` to filter by legacy IDs.

How to use query parameters and make successful requests:

- In our guides, we use HTTPS syntax for our example `GET` requests (rather than `cURL`, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the [main guide](#) and [API Overview and Authorisation guide](#).
- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
 - Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or `-` (multiple values, 'OR') or `!` ('NOT').
 - Use the required `_` (underscore) for any parameters listed with this prefix.
 - Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
 - If using the JSONP format (`_fmt=jsonp`) you **MUST** use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>comp</code> + <code>ctst</code>	Get seasonal statistics for a specific contestant in a competition (of a competition). Pass the contestant UUID to the <code>ctst</code> parameter, and competition UUID to the <code>comp</code> parameter. GET https://api.performfeeds.com/soccerdata/seasonstats/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&comp={competitionUuid}&ctst={contestantUuid}

Query parameter	Value and description
tmcl + ctst	Get seasonal statistics only for a specific contestant in a tournament calendar (of a competition). Pass the contestant UUID to the ctst parameter, and tournament calendar UUID to the tmcl parameter. <pre>GET https://api.performfeeds.com/soccerdata/seasonstats/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&tmcl={tournamentCalendarUuid}&ctst={contestantUuid}</pre>
detailed	Get an extended list of (detailed) seasonal statistics. Pass a value of no or yes (default) to the detailed parameter. Note: This parameter is valid only when combined with either the tmcl, comp or ctst parameters and an associated UUID (represented by {restOfQueryString} in the below URL path example) <pre>GET https://api.performfeeds.com/soccerdata/seasonstats/{outletAuthKey}/?_rt={mode}&_fmt={dataFormat}&detailed={yes no}&{restOfQueryString}</pre>
comp + ctst + detailed	Get an extended list of (detailed) seasonal statistics for a specific contestant in a competition (of a competition). Pass the contestant UUID to the ctst parameter, and competition UUID to the comp parameter, and yes (default) to the detailed parameter. <pre>GET https://api.performfeeds.com/soccerdata/seasonstats/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&comp={competitionUuid}&ctst={contestantUuid}&detailed={yes no}</pre>

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML sample - tmcl & ctst \(basic\)](#)
- [XML sample - tmcl & ctst \(detailed\)](#)
- [JSON sample - tmcl & ctst \(basic\)](#)
- [JSON sample - tmcl & ctst \(detailed\)](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▶<seasonStatistics> (XML only)

▶<competition>

▶<tournamentCalendar>

▶<contestant>

▶<stat>

▶<player>

[▶<stat>](#)

Statistic Types

Statistics are returned for the given contestant and for each player, and are delivered under the **stat** object. Stats are split into the following elements (where available):

- **contestant** (team)
- **player** (belonging to the given contestant/team)
- **currentTeamOnlyStat** (statistics for a player's current team)

Within these elements, each statistic is shown as a **stat** and its **name** and **value** will appear under it, if it is available.

Click on the buttons below to see the complete lists of contestant (team) and player statistics.

Contestant/Team Stats

[▶Contestant/Team stats - names and descriptions](#)

Player Stats

[▶Player stats - names and descriptions](#)